

Data Replication in a Single-Leader Environment

CENG465 — Principles of Data-Intensive Systems
Progress Presentation

Mert Karahan Doğukan Topçu

CENG465

May 5, 2026

Outline

- 1 Motivation & Scope
- 2 Why MongoDB?
- 3 Architecture
- 4 Setup Challenges
- 5 Schema Design
- 6 Implementation
- 7 Results
- 8 Conclusion & Next Steps

What Is This Project About?

Goal: Build and observe a **single-leader replication** system across two physical machines.

Required behavior:

- One node acts as **Leader (Primary)** — receives all writes
- One node acts as **Follower (Secondary)** — replicates from leader
- Measure **when** and **how quickly** writes propagate
- Log every operation with ordering metadata

Deployment: Two separate macOS machines on the same LAN — satisfies the project's *truly distributed* requirement.

Project Scope (Progress)

- ① **Environment Setup**
Replica set on two machines
- ② **Schema Design**
Replication-aware data model
- ③ **Replication Logging**
Operation logs with timestamps
- ④ **Insert / Update / Delete**
Visibility and delay tracking

Database Choice: MongoDB Replica Set

Key requirements from the project:

- Single-leader replication
- Observable update propagation
- Support for insert, update, delete
- Replication lag must be measurable

MongoDB Replica Set provides all of this natively:

- **Primary** node accepts writes
- **Secondary** applies operations from the **oplog**
- Replication is **asynchronous** — lag is observable
- `rs.status()` exposes health and lag in real time

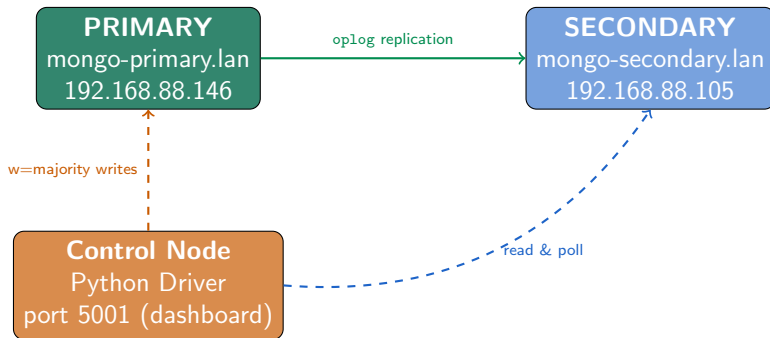
Raft-Inspired Metadata

Rather than implementing full Raft, we embed Raft-style tracking fields directly in our documents:

- `leader_term`
- `log_index`
- `operation_id`
- `version`

This allows analysis of **update ordering** and **visibility** without replacing MongoDB's built-in replication.

System Architecture



Replica set name: rs0 **Port:** 27017

Write concern: w="majority" — primary confirms before returning

Schema: items Collection

Stores application data with replication-aware tracking fields.

```
{
  "_id":           ObjectId,
  "key":           string,           # human-readable identifier
  "value":         object,          # application payload
  "version":       int,             # increments on every write
  "leader_term":   int,             # Raft-inspired term number
  "last_log_index": int,            # global operation order
  "last_operation_id": UUID string, # unique per write
  "last_updated":  ISODate,         # timestamp of last write on leader
  "deleted":       bool,            # soft-delete flag
  "created_by":    string           # client identifier
}
```

Why version?

Allows the control node to poll the secondary and detect *exactly* when a specific write becomes visible — without relying on timestamps alone.

Why soft delete?

Preserves the document on secondary for replication analysis. Hard deletes would remove the evidence of propagation timing.

Schema: operation_logs Collection

Every write produces one log entry. This collection is the primary source of evidence for replication behavior.

```
{
  "operation_id":      UUID string,
  "leader_term":       int,
  "log_index":         int,                # global ordering
  "operation_type":    "insert" | "update" | "delete",
  "target_id":         ObjectId,
  "leader_write_time": ISODate,            # write confirmed on primary
  "follower_visible_time": ISODate | null, # secondary returned correct version
  "replication_delay_ms": float | null,    # follower_visible - leader_write
  "version_before":    int | null,
  "version_after":     int,
  "status":            "visible_on_follower" | "timeout"
}
```

Key Insight

$\text{replication_delay_ms} = \text{follower_visible_time} - \text{leader_write_time}$

This is measured by polling the secondary every 10ms until `version == version_after`.

Implementation Overview

Control Node (Python)

- **config.py** — hosts, ports, timeouts
- **db.py** — primary & secondary connections
- **operations.py** — insert/update/delete with logging
- **app.py** — Flask web dashboard (port 5001)
- **test_replication.py** — 13 assertions
- **trace.py** — live change stream watcher

Write Flow (every operation)

- 1 Write to primary with `w="majority"`
- 2 Record `leader_write_time`
- 3 Poll secondary every 10ms
- 4 When version matches: record `follower_visible_time`
- 5 Compute `replication_delay_ms`
- 6 Insert into `operation_logs`

Timeout: 5000ms — if secondary does not show the update within 5 seconds, the log entry status is set to "timeout".

Replica Set Verification

After both machines were running, the replica set was initialized from the primary:

`rs.initiate()` **command:**

- `_id: "rs0"`
- Member 0:
`mongo-primary.lan:27017 (priority 2)`
- Member 1:
`mongo-secondary.lan:27017 (priority 1)`

Verified result

`mongo-primary.lan:27017` **PRIMARY**

`mongo-secondary.lan:27017` **SECONDARY**

Write concern: `w="majority"`

Every write operation uses `WriteConcern(w="majority")`, which means MongoDB waits for the primary *and* at least one secondary to acknowledge the write before confirming to the client.

Test Suite: 13/13 Passed

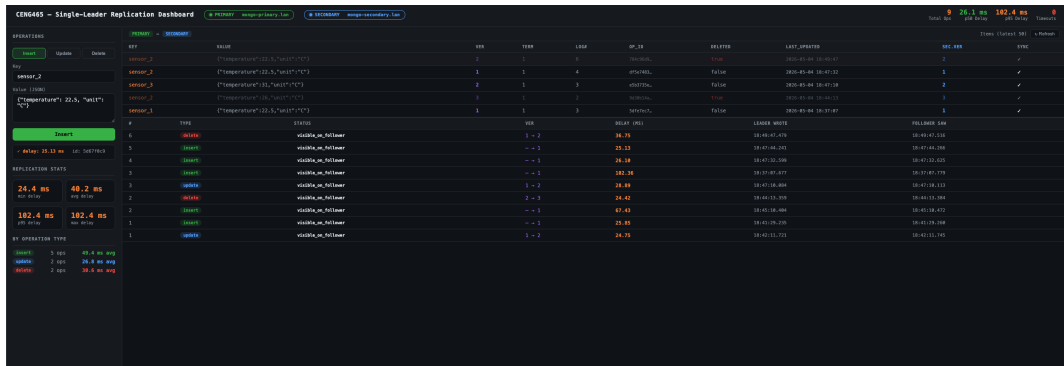
5 test categories:

- 1 **Basic CRUD** — insert, update, soft delete visible on secondary
- 2 **Burst writes** — 10 rapid inserts, all replicated
- 3 **Large payload** — 50KB document, data integrity check
- 4 **Sequential versions** — 5 updates, correct final version on secondary
- 5 **Monotonic versions** — secondary never returns a stale version

Result: 13/13 PASS

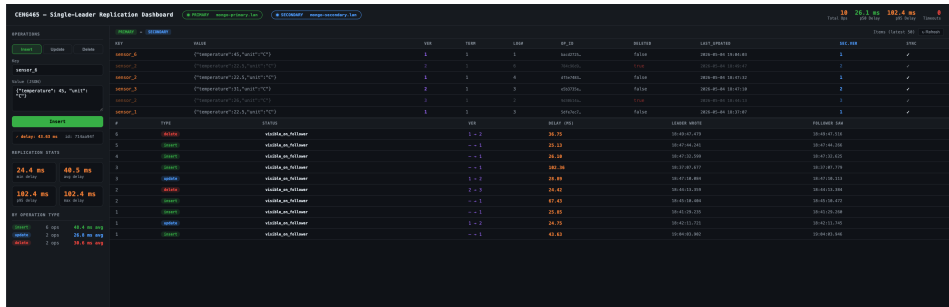
- Success rate: 100%
- 0 timeouts across all test cases
- Secondary always reflected correct version
- Soft delete flag propagated correctly
- Version monotonicity held across all reads

Web Dashboard — Overview



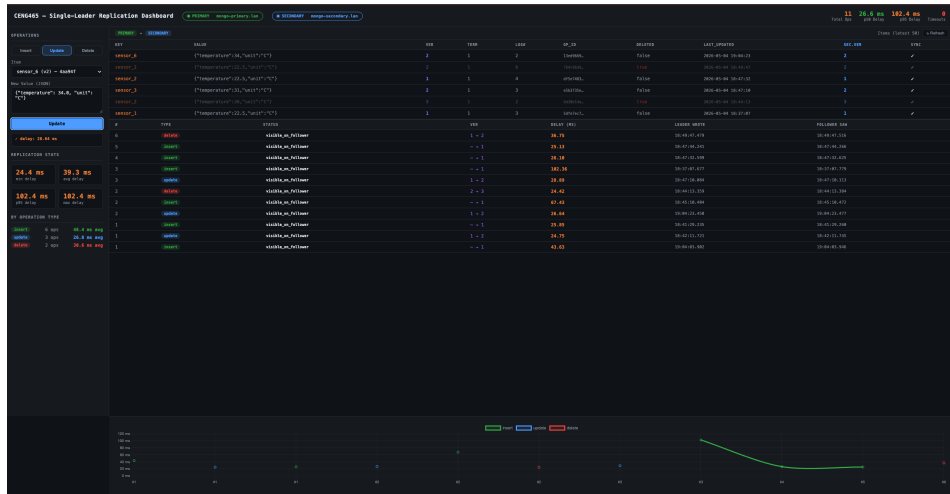
Live dashboard at <http://192.168.88.146:5001> — accessible from both machines simultaneously.

Web Dashboard — Insert Operation



- 1 Write sent to PRIMARY with w=majority
- 2 Secondary polled every 10ms until version=1 appears
- 3 replication_delay_ms logged with leader_write_time and follower_visible_time

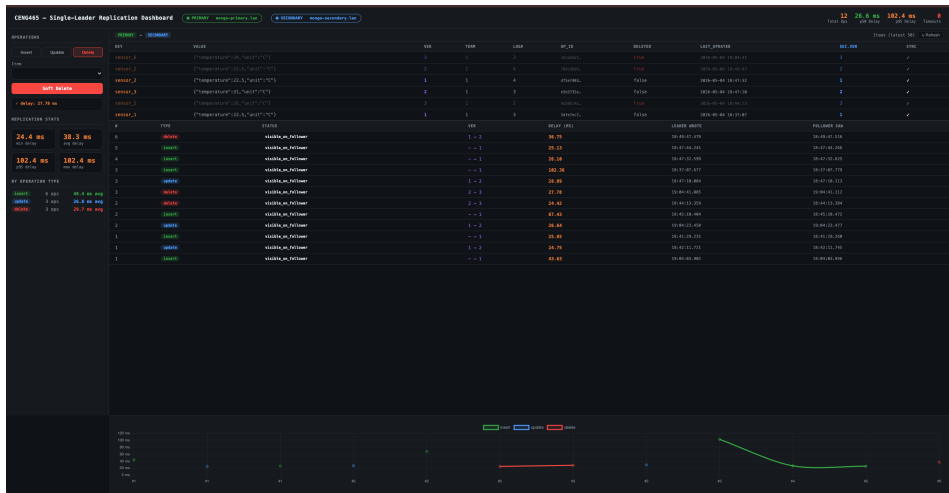
Web Dashboard — Update Operation



① New value sent to primary, version increments (1 → 2)

② Secondary polled until version matches

Web Dashboard — Delete Operation



① deleted: true flag written to primary, version still increments

② Document preserved on secondary for replication timing analysis

Conclusion & Next Steps

Completed (Progress Presentation):

- ✓ MongoDB Replica Set on 2 physical machines
- ✓ Replication-aware schema with version tracking
- ✓ Insert / Update / Soft Delete operations
- ✓ Replication delay measurement & logging
- ✓ 1000-op benchmark (100% success)
- ✓ 13/13 test assertions passing
- ✓ Live web dashboard (cross-machine)

Planned (Final Report):

- Eventual consistency experiment
- Monotonic reads experiment
- Read-after-write consistency
- Concurrent writes analysis
- Export logs as CSV/JSON
- Statistical analysis & graphs
- Final report & presentation PDF

Thank you.

Questions?